

Fundamentos de sistemas embebidos. Gpo. 2 Sem. 2026-2
317147531 – Sanchez Diaz Daniel
Documentación Proyecto Final Consola Retro SDK
Enfoque en la Emulación

1. Objetivo

Diseñar e implementar una consola de videojuegos retro embebida en una Raspberry Pi 4, capaz de emular sistemas NES, SNES y Game Boy Advance mediante el emulador mednafen, operando sobre Raspberry Pi OS Lite sin entorno gráfico. El sistema arranca de forma autónoma al recibir alimentación, presenta al usuario una galería de juegos navegable exclusivamente mediante un mando, permite la carga de nuevas ROMs a través de memoria USB de forma automática, y se apaga de manera segura desde el propio menú.

2. Lista de materiales.

Componente	Cantidad	Notas
Raspberry Pi 4 Model B (2 GB o más)	1	ARM Cortex-A72 64-bit
Tarjeta microSD (16 GB+, clase 10)	1	Sistema operativo y ROMs
Monitor o TV con entrada HDMI	1	Resolución mínima 720p
Cable micro-HDMI a HDMI	1	Incluido en kits oficiales
Fuente de alimentación 5 V / 3 A USB-C	1	Se recomienda fuente oficial
Gamepad USB compatible con SDL2	1	Ej. Xbox One, genérico USB HID
Memoria USB (FAT32 o exFAT)	1	Para carga de ROMs adicionales

Tabla 1. Hardware requerido

En cuanto al software se requiere:

- Raspberry Pi OS Lite (64-bit), sin entorno de escritorio
- Python 3.11 o superior
- pygame 2.x (python3-pygame)
- pyudev — bindings Python para libudev (python3-pyudev)
- mednafen — emulador multi-sistema (apt install mednafen)
- SDL2 con driver kmsdrm (libsdl2-2.0-0, libsdl2-dev)
- ROMs libres de derechos para NES, SNES y Game Boy Advance (mínimo 15)

3. Antecedentes teóricos

3.1 Sistemas embebidos y control de entrada

Un sistema embebido de entretenimiento como esta consola retro está diseñado para una función específica y opera de forma autónoma [1]. En este contexto, el gamepad es el único mecanismo de comunicación entre el usuario y el sistema: no existe teclado ni ratón. El estándar HID (Human Interface Device) define un protocolo USB genérico que expone los controles como botones (valores binarios), ejes analógicos (valores entre -1.0 y $+1.0$) y D-pad (valores discretos). SDL2, y por extensión pygame, abstrae estos dispositivos bajo una API uniforme independientemente del fabricante [2].

3.2 Modelo de eventos de pygame

pygame implementa un modelo orientado a eventos. Todos los eventos de entrada se insertan en una cola interna gestionada por SDL2. El programador debe vaciar esta cola regularmente con `pygame.event.get()` o mantenerla activa con `pygame.event.pump()`. Si la cola no se procesa, SDL2 deja de actualizar el estado de los dispositivos y el sistema operativo puede marcar la aplicación como 'no responde'.

Los eventos relevantes para este módulo son: `JOYBUTTONDOWN`, `JOYHATMOTION`, `JOYAXISMOTION` para el gamepad, `KEYDOWN` para el teclado, y `JOYDEVICEADDED` / `JOYDEVICEREMOVED` para la detección de gamepad en caliente (hot-plug).

3.3 Zona muerta en ejes analógicos

Los ejes analógicos raramente devuelven exactamente 0.0 en reposo por imprecisiones mecánicas del sensor (drift). Sin una zona muerta, el menú navegaría de forma errática sin que el usuario toque el control. Se define un umbral de 0.5 (50 % del recorrido) por debajo del cual el valor se considera cero [2].

3.4 mednafen y la gestión del framebuffer

Mednafen es un emulador de línea de comandos que soporta NES, SNES y GBA. Se lanza con el comando `mednafen -force_module <sistema> <ruta_rom>`. El flag `-force_module` evita que mednafen intente detectar automáticamente el sistema, lo que puede producir resultados incorrectos [9].

En Raspberry Pi OS Lite, KMS/DRM garantiza acceso exclusivo al framebuffer: solo un proceso puede tener el control del display en un momento dado [4]. Cuando pygame está activo tiene el framebuffer. Al lanzar mednafen sin liberarlo primero, mednafen falla con 'No available video device'. El protocolo de liberación y recuperación es: `pygame.display.quit()` → `sleep(1)` → `Popen()` → `wait()` → `sleep(0.5)` → `pygame.display.init()`.

3.5 subprocess.Popen vs subprocess.run

`subprocess.Popen()` lanza el proceso y devuelve inmediatamente un objeto que representa el proceso en ejecución, permitiendo interactuar con él (enviar señales, terminarlo). `subprocess.run()` lanza y bloquea hasta que termina. Se usa `Popen` porque la referencia al proceso se guarda en `self.proceso`, lo que permite que MonitorUSB lo termine con `proceso.terminate()` si se inserta un USB con ROMs mientras se está jugando [6].

3.6 Visión general del sistema

El sistema está compuesto por siete módulos que se instancian en `main.py`. La Tabla 2 muestra todos los módulos y cómo cada uno contribuye al sistema completo.

Módulo	Archivo	Función en el sistema
Pantalla	<code>pantalla.py</code>	Renderizado de toda la interfaz gráfica con pygame
Arranque	<code>arranque.py</code>	Secuencia de animación de logo al encender
Galeria	<code>galeria.py</code>	Loop principal; navega el catálogo y llama a <code>lanzar()</code>
Entrada	<code>entrada.py</code>	Captura de gamepad/teclado y mapeo a acciones lógicas

Emulador	emulador.py	Gestión del framebuffer y lanzamiento de mednafen
MonitorUSB	usb_monitor.py	Detección de USB; puede terminar mednafen si es necesario
Principal	main.py	Punto de entrada; instancia y conecta todos los módulos

Tabla 2. Módulos del sistema Consola Retro SDK

La interfaz gráfica (pantalla.py, arranque.py, galeria.py) maneja todo el renderizado. El monitoreo de USB (usb_monitor.py) corre en un hilo separado. main.py instancia y conecta todos los módulos pasando referencias entre ellos.

3.7 Interfaz gráfica con pygame y doble buffer

pygame es la biblioteca Python que gestiona todo el renderizado del sistema. Sigue el paradigma de doble buffer: todas las operaciones de dibujo se realizan sobre un buffer en memoria (back buffer), y solo cuando se llama a pygame.display.flip() el contenido se transfiere a la pantalla visible (front buffer). Esto elimina el parpadeo que se produciría si se dibujara directamente sobre el buffer visible [3]. La clase Pantalla encapsula toda esta lógica: ningún otro módulo llama a pygame directamente, lo que permite cambiar la biblioteca de renderizado sin modificar el resto del sistema.

La animación de arranque utiliza interpolación de alpha: el método Surface.set_alpha(valor) acepta valores entre 0 (transparente) y 255 (opaco). En cada frame se genera una copia del logo con Surface.copy() para no modificar la superficie original, se actualiza su opacidad y se dibuja. El menú de galería usa una ventana deslizante para mostrar solo 14 ROMs a la vez, centrada en el ítem seleccionado: inicio = max(0, indice - V // 2), fin = min(total, inicio + V) [3].

3.8 Arranque automático, udev y monitoreo de USB

El arranque automático se configura mediante systemd, el gestor de servicios estándar en Raspberry Pi OS. El archivo consola-retro.service indica qué programa ejecutar, con qué usuario y qué hacer si el proceso falla (Restart=always). La directiva ConditionPathExists=!/boot/NO_CONSOLA permite deshabilitar la consola creando ese archivo, lo que facilita el acceso al sistema para mantenimiento sin modificar la configuración del servicio [7].

La detección de USB se realiza mediante pyudev, que escucha eventos del kernel en lugar de revisar periódicamente el contenido de /dev (polling). El hilo de monitoreo queda bloqueado en monitor.poll() consumiendo 0% de CPU hasta que el kernel genera un evento de tipo 'add' en el subsistema 'block'. La comunicación entre el hilo de monitoreo y el loop principal de la galería se protege con threading.Lock para evitar condiciones de carrera sobre la variable compartida _usb_nuevo [8].

4. Advertencias y cuidado de componentes

4.1 Seguridad eléctrica

- La Raspberry Pi 4 opera a 5 VCC. Por lo mismo no se recomienda que pases de ese voltaje, siempre utiliza el cargador que viene con la Raspberry.
- Al conectar el mando por USB se recomienda tener mucho cuidado para no dañar los puertos de la Raspberry.

4.2 Cuidado de la tarjeta microSD

- Siempre apague el sistema a través del menú de confirmación (botón B y luego A) para que el kernel desmonte correctamente, ya que si apaga jalando del cable que conecta a la corriente puede corromper los archivos.
- Se recomienda que las ROMs que se deseen cargar lo hagan por medio de la USB ya que ya hay un sistema incluido que evita hacerlo por otros medios y con riesgos de que se corrompan los datos o se pierdan, además de que los acomoda automáticamente dependiendo de la consola.

5. Descripción de los módulos de entrada.py y emulación.py

El subsistema de entrada y emulación está compuesto por dos módulos: entrada.py y emulador.py. La Tabla 3 muestra el mapeo completo de acciones lógicas.

Acción lógica	Botón gamepad	Tecla teclado	Descripción
ARRIBA	Hat ↑ / Eje Y < -0.5	↑	Navegar hacia arriba en el menú
ABAJO	Hat ↓ / Eje Y > +0.5	↓	Navegar hacia abajo en el menú
IZQUIERDA	Hat ← / Eje X < -0.5	←	Reservado para uso futuro
DERECHA	Hat → / Eje X > +0.5	→	Reservado para uso futuro
CONFIRMAR	Botón 0 (A)	Enter	Seleccionar juego / confirmar
ATRAS	Botón 1 (B)	Escape	Cancelar / menú anterior
INICIO	Botón 7 (Start)	F1	Confirmar apagado del sistema

Tabla 3. Mapeo de acciones lógicas a controles físicos

Consola	Flag mednafen	Extensiones	Nombre completo
NES	-force_module nes	.nes	Nintendo Entertainment System
SNES	-force_module snes	.smc / .sfc	Super Nintendo Entertainment System
GBA	-force_module gba	.gba	Game Boy Advance

Tabla 4. Consolas soportadas y configuración de mednafen

Método	Clase	Descripción
leer()	Entrada	Lee eventos pygame y devuelve una acción lógica
_tecla_a_accion(tecla)	Entrada	Mapea código de tecla a acción lógica
_hat_a_accion(valor)	Entrada	Convierte valor del D-pad a acción lógica

<code>_eje_a_accion(eje, valor)</code>	Entrada	Convierte eje analógico a acción (umbral 0.5)
<code>lanzar(console, nombre, ruta)</code>	Emulador	Orquesta el ciclo completo de lanzamiento del juego
<code>_mostrar_controles(console, rom)</code>	Emulador	Muestra imagen de controles con texto parpadeante
<code>_mostrar_error(mensaje)</code>	Emulador	Despliega mensaje de error durante 3 segundos

Tabla 5. Métodos de los módulos *entrada.py* y *emulador.py*

5.1 entrada.py

La clase Entrada actúa como traductor entre el hardware físico y el resto del sistema. En lugar de que la galería conozca los botones específicos del mando, trabaja con acciones lógicas (ACCION_ARRIBA, ACCION_CONFIRMAR, etc.) definidas como constantes de cadena. Si se cambia el modelo de mando, solo se modifica este archivo sin tocar ningún otro módulo.

5.2 emulador.py

La clase Emulador gestiona todo el ciclo de vida de una sesión de juego: desde la pantalla previa de controles hasta el regreso al menú principal. Oculta la complejidad de la gestión del framebuffer al resto del sistema; la galería simplemente llama a `emulador.lanzar()` y recupera el control cuando el juego termina.

5.2.1 Pantalla de controles

Antes de entrar a mednafen se muestra la imagen de la distribución de botones para la consola seleccionada. El nombre del juego aparece en dorado. El mensaje “Presiona A para jugar | Escape para volver” parpadea cada 500 ms usando `time.time()` en lugar de temporizadores de pygame, para evitar dependencias del display durante la transición crítica de framebuffer.

5.3 Módulos de interfaz gráfica

Los módulos de interfaz son los responsables de todo lo visual del sistema. `pantalla.py` define la clase Pantalla, que inicializa pygame en modo pantalla completa con el driver kmsdrm, define tres tamaños de fuente monoespaciada (grande 52 pt, normal 32 pt, pequeña 20 pt) y expone métodos como `limpiar()`, `dibujar_texto()`, `dibujar_imagen()`, `reproducir_sonido()` y `actualizar()`. `arranque.py` implementa la secuencia de presentación al encender: carga el logo, reproduce el sonido de moneda, ejecuta el fade-in durante 1.5 segundos, mantiene el logo visible 2 segundos y ejecuta el fade-out. `galeria.py` contiene el loop principal del sistema a 30 fps: escanea las carpetas de ROMs, dibuja el menú con ventana deslizante y gestiona la confirmación de apagado mediante `sudo shutdown -h now` [3].

5.4 Módulos de sistema e integración

`main.py` es el único archivo que se ejecuta directamente, instancia los módulos en el orden correcto (Pantalla → Entrada → MonitorUSB → Emulador → Galería → Arranque), pasa las referencias entre ellos y al terminar llama a `pantalla.cerrar()` en un bloque `finally`. `usb_monitor.py` implementa la clase MonitorUSB, que corre en un hilo daemon separado para no bloquear la interfaz. Sus métodos principales son: `hay_nuevo_usb()` (consultado en cada frame por la galería), `copiar_roms_de_usb()` (copia solo archivos nuevos comparando nombres en minúsculas para evitar duplicados) y `_desmontar()` (libera el USB de forma segura al terminar). Si pyudev no está disponible, el sistema usa polling cada 2 segundos como alternativa [8].

6. Instalación del sistema

El sistema se instala ejecutando un único comando desde la terminal SSH de la Raspberry Pi. El script `instalar.sh` automatiza todos los pasos necesarios para tener la consola funcionando desde cero en una instalación limpia de Raspberry Pi OS Lite:

```
sudo bash -c "$(curl -fsSL
https://raw.githubusercontent.com/nasaruke/SDK_RetroGames/main/src/instalar.sh)"
```

Los pasos que ejecuta el script en orden son:

1. Instala dependencias.
2. Descarga el proyecto.
3. Crea carpetas de ROMs.
4. Configura Mednafen.
5. Da permisos al usuario.
6. Configura USB y apagado.
7. Crea el servicio de arranque automático.
8. Oculta mensajes de inicio.
9. Reinicia la Raspberry.

El script detecta automáticamente el usuario real, aunque se ejecute con `sudo`, instala todas las dependencias, descarga el código fuente y los assets desde GitHub, configura el servicio para el arranque automático y reinicia la Raspberry Pi al finalizar. El proceso completo tarda aproximadamente 10 a 15 minutos dependiendo de la velocidad de la conexión a internet. Una vez instalado, el sistema arranca automáticamente cada vez que se enciende la Raspberry Pi.

7. Desarrollo de los módulos

7.1 método `leer()`

```
def leer(self) -> str:
    pygame.event.pump()
    for evento in pygame.event.get():
        if evento.type == pygame.QUIT:
            return ACCION_ATRAS
        if evento.type == pygame.KEYDOWN:
            return self._tecla_a_accion(evento.key)
        if evento.type == pygame.JOYDEVICEADDED:
            self.joystick =
pygame.joystick.Joystick(evento.device_index)
            self.joystick.init()
        if evento.type == pygame.JOYDEVICEREMOVED:
            self.joystick = None
        if evento.type == pygame.JOYBUTTONDOWN:
            return self.MAPA_BOTONES.get(evento.button,
ACCION_NINGUNA)
        if evento.type == pygame.JOYHATMOTION:
            return self._hat_a_accion(evento.value)
        if evento.type == pygame.JOYAXISMOTION:
            return self._eje_a_accion(evento.axis, evento.value)
    return ACCION_NINGUNA
```

7.1.1 Conversión de ejes analógicos con zona muerta

```
def _eje_a_accion(self, eje: int, valor: float) -> str:
    if abs(valor) < self.UMBRAL_EJE: # zona muerta 50%
        return ACCION_NINGUNA
    if eje == 0: # eje horizontal
        return ACCION_DERECHA if valor > 0 else ACCION_IZQUIERDA
    if eje == 1: # eje vertical
        return ACCION_ABAJO if valor > 0 else ACCION_ARRIBA
    return ACCION_NINGUNA
```

7.2 método lanzar()

```
def lanzar(self, consola, nombre_rom, ruta_rom):
    if not self._mostrar_controles(consola, nombre_rom):
        return
    comando = [ejecutable] + flags + [ruta_rom]
    try:
        pygame.mixer.stop()
        pygame.display.quit() # liberar framebuffer
        time.sleep(1)
        entorno = os.environ.copy()
        entorno['SDL_VIDEODRIVER'] = 'kmsdrm'
        entorno['SDL_AUDIODRIVER'] = 'alsa'
        self.proceso = subprocess.Popen(comando, env=entorno)
        self.proceso.wait()
    except FileNotFoundError:
        print('Error: mednafen no encontrado')
    finally:
        self.proceso = None
        time.sleep(0.5)
        pygame.display.init() # recuperar framebuffer
        pygame.mixer.init()
        self.pantalla.superficie = pygame.display.set_mode(
            (self.pantalla.ancha, self.pantalla.alto),
            pygame.FULLSCREEN
        )
        pygame.mouse.set_visible(False)
```

7.3 Pantalla de controles con parpadeo

```
parpadeo = True
ultimo_flip = 0
while True:
    ahora = time.time()
    if ahora - ultimo_flip > 0.5:
        parpadeo = not parpadeo
        ultimo_flip = ahora
        if parpadeo:
            self.pantalla.dibujar_texto(
                '>> Presiona A para jugar | Escape para volver
<<',
                ..., color=(50, 255, 50)
            )
            self.pantalla.actualizar()
            reloj.tick(30)
```

8. Integración del subsistema

Los módulos `entrada.py` y `emulador.py` se instancian en `main.py` y se pasan por referencia a la clase `Galeria`. La galería llama a `entrada.leer()` en cada iteración de su loop principal para obtener la acción del usuario. Cuando el usuario selecciona una ROM, la galería llama a `emulador.lanzar()` que bloquea hasta que el juego termina y luego devuelve el control al loop de la galería.

La referencia al objeto `Emulador` también se pasa al `MonitorUSB`, que la usa para terminar el proceso de `mednafen` (`self.proceso.terminate()`) si se inserta un USB con ROMs mientras se está jugando. Esto demuestra cómo el diseño basado en referencias a objetos permite la comunicación entre módulos sin acoplamiento directo.

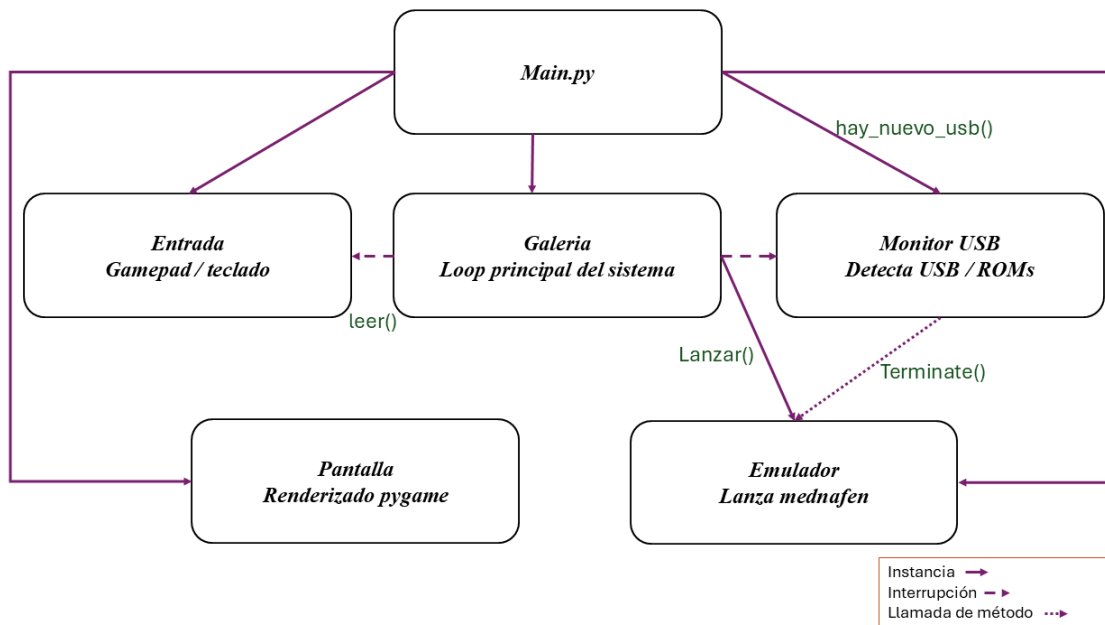


Figura 1. Diagrama de interacción entre *Galeria*, *Entrada*, *Emulador* y *MonitorUSB*

9. Solución de problemas comunes

9.1 mednafen no inicia: 'No available video device'

Verificar que `pygame.display.quit()` se llama antes de `subprocess.Popen()` y que se espera al menos 1 segundo entre ambas llamadas. Verificar también que `SDL_VIDEODRIVER=kmsdrm` está en el entorno del subprocesso.

9.2 El gamepad no es detectado al arrancar

Verificar que el usuario pertenece al grupo `input` con `groups $USER`. Si el gamepad se conectó después del arranque, `pygame` debería detectarlo automáticamente mediante `JOYDEVICEADDED`. Si no es así, verificar con `lsusb` y `dmesg | tail`.

9.3 El menú navega solo (drift de ejes)

El umbral de zona muerta puede ser insuficiente para el gamepad utilizado. Aumentar `UMBRAL_EJE` en la clase `Entrada` de 0.5 a 0.6 o 0.7 según sea necesario.

9.4 Pantalla en negro al regresar de mednafen

Aumentar `time.sleep(0.5)` en el bloque `finally` del método `lanzar()` a 1.0 segundo para dar más tiempo a que `mednafen` libere el `framebuffer`.

10. Evidencias

Repositorio del proyecto: https://github.com/nasaruke/SDK_RetroGames

Video demostrativo: <https://youtu.be/pv0NvIFaCqE>

11. Conclusiones

El proyecto nos ayudó a corroborar todos los aprendizajes en lecciones pasadas de las practicas, sobre todo de la de kiosco multimedia ya que esa fue un inicio de como podíamos esconder las llamadas y el arranque de la raspberry que fue donde creo este proyecto nos atoro más, puesto que por más que intentábamos ocultarlas no nos fue posible hacerlo en un 100%. Otras dificultades por las que atravesamos fueron en el arranque y salida de `Mednafen` pues llego un punto del proyecto en el que al intentar correr un juego desde el menú no había ningún problema, pero, cuando se necesitaba que reingresara al menú nos mandaba directo a la línea de comando.

Con esto puedo decir que se cumplió el poder crear un sistema embebido desde cero y que encontré muy asombroso ya que, con hardware muy accesible y robusto, con la integración de una interfaz para el usuario con una gestión automática de USBs y ROMs que facilita la carga de videojuegos y que esto no obstruye con una experiencia fluida dentro de los juegos.

En cuanto a la emulación se puede aparentar que solo con la descarga del `Mednafen` o cualquier otro emulador y escribir el comando para arrancar la rom es todo, sin embargo, el crear un entorno que nos ayude a facilitar esto para que no sea necesario el escribir o configurar cada juego que juguemos, sino, mas bien automatizar todos estos pasos para ofrecer una mejor experiencia de juego.

12. Cuestionario

1. ¿Por qué se utiliza doble buffer (`display.flip`) en lugar de actualizar la pantalla píxel a píxel?
2. ¿Qué ventaja tiene encapsular todas las llamadas a `pygame` dentro de la clase `Pantalla` en lugar de llamar a `pygame` directamente desde cada módulo?
3. ¿Por qué se define un umbral de 0.5 para los ejes analógicos en lugar de detectar cualquier valor distinto de cero? ¿Qué problema resuelve esto en la práctica?
4. Describe el problema del `framebuffer` compartido entre `pygame` y `mednafen`. ¿Por qué es necesario llamar a `pygame.display.quit()` antes de lanzar `mednafen` y cómo se recupera la pantalla al regresar?
5. ¿Qué función cumple el `threading.Lock` en la clase `MonitorUSB`? ¿Qué condición de carrera podría ocurrir si se eliminara el lock?
6. ¿Por qué se utiliza `pyudev` para detectar la inserción de USB en lugar de revisar periódicamente el contenido de `/dev`? Describe las ventajas del enfoque basado en eventos sobre el polling.

13. Bibliografía

[1] E. A. Lee y S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2.a ed. Cambridge, MA: MIT Press, 2017.

- [2] Raspberry Pi Foundation, "Raspberry Pi OS Documentation," 2024. [En línea]. Disponible en: <https://www.raspberrypi.com/documentation/>
- [3] pygame Developers, "pygame Documentation," 2024. [En línea]. Disponible en: <https://www.pygame.org/docs/>
- [4] SDL Community, "SDL2 Wiki," 2024. [En línea]. Disponible en: <https://wiki.libsdl.org/SDL2/>
- [5] M. Kerrisk, The Linux Programming Interface. San Francisco, CA: No Starch Press, 2010.
- [6] Python Software Foundation, "threading — Thread-based parallelism," Python 3.11. Disponible en: <https://docs.python.org/3/library/threading.html>
- [7] Lennart Poettering y Kay Sievers, "systemd System and Service Manager," freedesktop.org, 2024. Disponible en: <https://www.freedesktop.org/wiki/Software/systemd/>
- [8] pyudev Contributors, "pyudev Documentation," 2024. Disponible en: <https://pyudev.readthedocs.io/>
- [9] Mednafen Team, "Mednafen Documentation," 2024. Disponible en: <https://mednafen.github.io/documentation/>